

Embedded Real-Time Systems

Recap

Henri Heyden

stu240825

Kiel University

Summer Term 2026

Disclaimer

I do not guarantee this summary to contain every part of the lecture, as it is only supposed to summarize the things I personally found the most important when going through the slides.

This document was compiled on September 17, 2026.

Introduction

The Embedded Real-Time Systems lecture of Kiel University by Reinhard von Hanxleden is based on *Introduction to Embedded Systems—A Cyber-Physical Systems Approach* by Edward Lee and Sanjit Seshia¹. Embedded systems are computers whose main purpose is the interaction with a physical process. This interaction is commonly implemented using sensors, actuators, and physical dynamics. A real-time system is broadly defined by the notion, that not only the correctness of the output of a machine matters, but also the timing in which they arrive.

In this book and lecture it is postulated, that embedded systems should be created using an iterative process, which is called *Modeling, Design, Analysis*.

Modeling The process of gaining a deeper understanding of a system through environmental imitation. Models specify *what* a system does.

Design The structured creation of artifacts. Designs specify *how* a system does what it does.

Analysis The process of gaining a deeper understanding of a system through dissection. Analysis specifies *why* a system does what it does, or fails to do so.

This approach has the important benefit, that we can implement mechanisms to compile embedded systems software based on a mathematical model of the system and environment. The compiler can perform a model analysis during translation.

¹https://ptolemy.berkeley.edu/books/leeseseshia/releases/LeeSeshia_DigitalV2_3.pdf

Contents

1	Modeling Dynamic Behaviours	7
1.1	Continuous Dynamics	7
1.2	Discrete Dynamics	9
1.3	Hybrid Automata	10
1.4	State Machine Compositions	11
2	Design of Embedded Systems	13
3	Analysis and Verification	15

Chapter 1

Modeling Dynamic Behaviours

In this chapter we will look at different dynamic mechanisms to model using a mathematical model based on so-called *actors* and *signals*.

1.1 Continuous Dynamics

Continuous dynamics directly mimic continuous physical processes. An example where continuous dynamics are used to control an embedded system is a proportional integral differential (PID) controller in a helicopter which is used to cancel out the torque generated by the main rotor using the side rotor.

Generally, the dynamical models in this chapter act on functions between time and some value.

Definition 1.1.1 (General Physical Axioms). In the model we define position and angle as functions on time.

$$\begin{aligned}x &: \mathbb{R} \rightarrow \mathbb{R}^3 \\ \theta &: \mathbb{R} \rightarrow \mathbb{R}^3\end{aligned}$$

From this we can obtain the (angular) velocity and acceleration as the respective first and second differential.

$$\ddot{x} = \partial \dot{x} = \partial \partial x$$

If we add a constant mass $M \in \mathbb{R}$, we can furthermore obtain force by Newton's second law.

$$F(t) = M \cdot \ddot{x}(t)$$

If we have a radius $r \in \mathbb{R}$ and an inertia tensor $I : \mathbb{R} \rightarrow \mathbb{R}^{3 \times 3}$, torque can be found for a point-mass using either

$$T(t) = r \cdot F(t)$$

or

$$T(t) = \partial \left(I(t) \cdot \dot{\theta} \right).$$

For one dimension and static inertia tensor, this boils down to

$$T_y(t) = I_{yy} \cdot \ddot{\theta}_y(t)$$

which can be written in integral form

$$\dot{\theta}(t) = \dot{\theta}(0) + 1/I_{yy} \int_0^t T_y(\tau) d\tau.$$

These physical processes are computationally hard to model, as a computer can not possibly predict the future, or solve complex integrals to direct equations. This is why in the *actor model of systems*, the model calculates general feedback integrals by using approximations.

Definition 1.1.2 (Actor Model of Systems). In continuous dynamics, the actor model of systems is defined by using so-called *actors*, which are functions on functions of time, which we call *signals*. For example

$$S : (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$$

is an actor.

As already mentioned, a computer can not predict the future, but this model helps by keeping the history in mind. The *integrator* actor $I : (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ can be thus defined.

$$I(x)(t) = x(0) + \int_0^t \dot{x}(\tau) d\tau$$

Here the actor has a parameter $x(0)$, the initial value, and produces a value function $x : \mathbb{R} \rightarrow \mathbb{R}$ based on initial value and acceleration $\dot{x}$, which is the input of the actor.

An actor can also have multiple signals as inputs, see $\oplus : (\mathbb{R} \rightarrow \mathbb{R})^2 \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ with

$$\oplus(x, y)(t) = x(t) + y(t).$$

1.2 Discrete Dynamics

In discrete dynamics we do not look at signals which provide input at every moment, but which are only *present* sometimes.

Definition 1.2.1 (Pure and discrete signals). A signal is called *pure* if it is of form $\mathbb{R} \rightarrow \{\text{absent}, \text{present}\}$ or $\mathbb{R} \rightarrow \{\text{absent}\} \cup S$ for some set S . We define a pure signal x to be *discrete* iff

$$\exists f \in \mathbb{Z}^{\{t \in \mathbb{R} | x(t) \neq \text{absent}\}}, f \text{ is injective} : \forall t_1, t_2 \in T : t_1 < t_2 \Rightarrow f(t_1) < f(t_2)$$

In other words, a signal x is discrete iff there exists an one-to-one function on the set of all moments, where x was not absent to $\mathbb{Z}$, which is order-preserving. All elements of this set have to be ordable on $\mathbb{Z}$. Alternatively, if we assume, that there exists a minimal or first time in which x was present, the function can also map to $\mathbb{N}$. Note, that countability does not imply discreteness!

We call an actor on discrete signals discrete, iff the output signal is also discrete. Note, that this is not trivial, the composition of two discrete signals does not have to be discrete.

Reactions of a discrete system are triggered by the environment in which the actor operates—it *reacts*. In the case that an actor produces an output, which happens on reaction, we call this output *valuation of type V* , where V is the domain of the domain of the actor.

As mentioned, we consider a discrete actor to react if, but not only if it changes its internal state, this can be done in different ways. Final state machines (FSM) can be used as a *refinement* of a discrete actor to define its behaviour.

Definition 1.2.2 (Final State Machines). FSMs are mathematical constructions consisting of *states*, *transitions*, and an *initial* state. A FSM starts in the initial state. Transitions are of form $s, g \rightarrow s', o$, where s is the state the transition starts in, g is the *guard* of the transition, s' is the state the transition ends in, and o is the (optimal) output valuation the FSM produces, when the transition is taken.

A FSM which is the refinement of a discrete actor changes its state if a guard of a possible transition is true. The actor produces a value, if the transition it takes has an output valuation, else it is absent.

The guard of a transition can be based on input signals. E.g., $p_1 \wedge \neg p_2 \vee p_3 = 3$ is true, when p_1 is present, and p_2 is absent, or when p_3 is present with value 3.

A transition can be set *default*, which means it is enabled if no non-default transition is enabled. This is highlighted by using a dotted line. Every FSM defined implicit default self transitions. I.e., if no transition is enabled, the FSM neither reacts nor changes its state—any present input is disregarded, but the system is still well defined.

It is also possible to define non-deterministic transitions, which is indicated through a red line. These transitions enable multiple transitions to be taken at the same time. If a FSM contains a non-deterministic transition, it is called a non-deterministic FSM. In this case, the set of resulting states becomes the permutation set of all possible states.

We extend the FSM model using the following model.

Definition 1.2.3 (Extended State Machines). Extended state machines (ESMs) add more features in comparison to FSMs. The extension brings variables $v \in \mathcal{V}$ and enables transitions to use *set actions* $v := \sigma$, where σ is the new value for v . σ can be any right-hand side of an output action—it can thus also be based on input signals.

We also define some basic terms on SMs.

Definition 1.2.4. A transition is called *stuttering*, iff it is enabled and produces an absent output. A *behaviour* is a sequence of non-stuttering steps. A *trace* is the record of inputs, states, and outputs in a behaviour. A *computation tree* is a graphical representation of all possible traces.

SMs provide a way to mathematically represent the environment of the system and the system itself. They can be used for designing requirements and analysing the safety of the system.

Currently we do not have defined, *when* a reaction occurs, i.e., when to read the next input values for the signals. This is vitally important, as we can define ESMs with a default self transition, which increases the value of a variable—the value becomes dependent on the *rate* of reactions, which is currently undefined. We will go over this notion of time in ??.

1.3 Hybrid Automata

In sections 1.1 and 1.2 we defined continuous and discrete automata, in this section we will define *hybrid automata*, which combine both approaches by refining individual states with continuous actors.

In hybrid automata, signals do not have to be discrete—they are allowed to be continuous-time signals. This again begs the question *when* a reaction happens. In this model, we define the actor to react *asap*—which depends on the simulation speed, when evaluating the model.

For example, we can define a continuous actor using a hybrid model which contains one state s with variable $v = 0$ and default transition $s, \text{true} \rightarrow s, v, \dot{v} := 1$. This hybrid automata defines a continuous actor with no input signal, which outputs the linear function.

Definition 1.3.1 (Hybrid Automata). We formally define hybrid automata to be ESMs with continuous input and output signals. Moreover, it is allowed to depend the value of a variable in *continuous time*, by setting derivatives of variables in a set action. Reactions occur as soon as possible.

Hybrid automata also define refinements to states which are themselves continuous actors.

1.4 State Machine Compositions

It is possible to combine different SM-refined (discrete) actors together on a top level. There are four different compositions with different semantics.

1. Synchronous side-by-side composition: Multiple SMs in parallel with reaction only possible when all SMs react.
2. Asynchronous side-by-side composition: Multiple SMs in parallel with non-deterministic reaction when any SM can react.
3. Cascade composition: Multiple SMs chained together with output signals as input signals for another SM.
4. Feedback composition: Recursive connection of a SM which sets input signals to own output signals.

For the following definitions, we will look at compositions between two SM actors A and B with states S_A, S_B , input valuations (or guards) i_A, i_B , output signals o_A, o_B , initial states I_A, I_B , and update function $u_A : S_A \times i_A \rightarrow S_A \times o_A$, for B analogous, to represent the transitions.

Definition 1.4.1 (Synchronous Side-by-side Composition). In synchronous side-by-side compositions for SMs A and B , we define the set of states as $S := S_A \times S_B$, the inputs $i := i_A \times i_B$, outputs $o := o_A \times o_B$, initial state $I := (I_A, I_B)$, and update function $u : S \times i \rightarrow S \times o$,

$$u((s_A, s_B), (v_A, v_B)) = ((s'_A, s'_B), (v'_A, v'_B)),$$

with $(s'_A, v'_A) = u_A(s_A, v_A)$ and $(s'_B, v'_B) = u_B(s_B, v_B)$.

In other words, we just concatenate all input, states, outputs, and derive next output/state by calculating the next state and output for both SMs, and combining them.

This means that, even though A and B may have one input pair, for which A reacts and B produces an absent value, the combined SM reacts. This is because B reacts without producing a value—the reactions are triggered by the environment.

Definition 1.4.2 (Asynchronous Side-by-side Composition). In the asynchronous side-by-side composition, the environment chooses if either A or B react¹. This is called *interleaving semantics*. Thus we have the same definition as in the synchronous version, but with the rule that the update function $u : S \times i \rightarrow S \times o$ is defined as

$$u((s_A, s_B), (v_A, v_B)) = \begin{cases} ((s'_A, s_B), (v'_A, \text{absent})) & \text{when } A \text{ reacts,} \\ ((s_A, s'_B), (\text{absent}, v'_B)) & \text{when } B \text{ reacts.} \end{cases}$$

This is realized using non-deterministic compositions when calculating the composite actor.

¹ It is also possible, that both actors react. These semantics are not discussed in this model.

Definition 1.4.3 (Cascade Composition). The cascade composition of A and B is considered to be the simultaneous and instantaneous reaction of both actors, where the output signal of A is the input signal of B . This is of course only possible if the expression *type checks*, i.e., when $o_A \subseteq i_B$.

To construct the composition machine, first form the state space as the cross product of the state spaces of A and B , and then determine which transitions are taken under what conditions. It is important to remember that the transitions are simultaneous, even when one logically causes the other.

Chapter 2

Design of Embedded Systems

Chapter 3

Analysis and Verification